

mikroflow — запуск на продакшене

Сбор NetFlow с MikroTik → PostgreSQL

2026-07-03

Что это

mikroflow собирает с роутера MikroTik информацию о соединениях (кто и куда подключался) и складывает в PostgreSQL для анализа.

- **Одна строка на соединение**: оба направления NetFlow свёрнуты на LAN-устройство (`device_ip`) ↔ внешний адрес (`remote_ip`).
- **Имя устройства** — из аренд DHCP; **MAC** — из DHCP и ARP (покрывает и статические хосты); можно задать **свои имена** по MAC (справочник).
- **Домен назначения** — reverse-DNS.
- Сырые флоу хранятся ~14 дней, почасовые агрегаты — 6 месяцев.
- Разворачивается через Docker Compose на Linux.
- Финальный анализ — по SQL-вьюхе `v_connections`.

Стек из трёх сервисов: `postgres`, `collector` (приём NetFlow по UDP), `worker` (синк DHCP/ARP, reverse-DNS, агрегация, ретеншн).

1. Подготовить сервер

Сервер — Linux с Docker.

```
# Docker + Compose plugin (Ubuntu/Debian)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER      # перелогиниться после этого
docker compose version             # проверка
```

2. Доставить код на сервер

Код в репозитории github.com/Aleksandovm/mikroflow, ветка `master`. Каталог `/opt` принадлежит `root`, поэтому сначала создай папку и передай её своему пользователю:

```
sudo mkdir -p /opt/mikroflow
sudo chown "$USER":"$USER" /opt/mikroflow
```

Если репозиторий публичный — клонируй без токена:

```
git clone https://github.com/Aleksandovm/mikroflow.git /opt/mikroflow
cd /opt/mikroflow
```

Если репозиторий приватный — нужен Personal Access Token (scope `repo`, создаётся на github.com/settings/tokens) в URL:

```
git clone https://<TOKEN>@github.com/Aleksandovm/mikroflow.git /opt/mikroflow
cd /opt/mikroflow
```

Обновление кода в будущем — просто `git pull` в `/opt/mikroflow`.

3. Настроить `.env` (ОБЯЗАТЕЛЬНО смени пароли)

```
cd /opt/mikroflow
cp .env.example .env
nano .env
```

Заполни:

```
MIKROFLOW_ROUTER_HOST=192.168.88.1      # IP твоего MikroTik
MIKROFLOW_ROUTER_USER=netflow           # read-only API-юзер (см. шаг 4)
MIKROFLOW_ROUTER_PASSWORD=<сильный_пароль>
MIKROFLOW_ROUTER_PORT=8728
MIKROFLOW_RAW_RETENTION_DAYS=14
MIKROFLOW_HOURLY_RETENTION_DAYS=180
```

Важно про пароль БД. В `docker-compose.yml` пароль Postgres по умолчанию `mikroflow`. Для прода поменяй его в двух местах — `POSTGRES_PASSWORD` у сервиса `postgres` и в `MIKROFLOW_DB_DSN` у `collector` и `worker`.

4. Настроить MikroTik

В терминале роутера:

```
# экспорт потоков на сервер
/ip traffic-flow set enabled=yes interfaces=all
/ip traffic-flow target add address=<IP_СЕРВЕРА>:2055 version=9

# read-only пользователь для чтения аренд DHCP и ARP через API
/user group add name=netflow-ro policy=api,read,test
/user add name=netflow group=netflow-ro password=<тот_же_пароль_что_в_.env>
/ip service enable api
```

Политики `read` достаточно и для аренд DHCP, и для таблицы ARP.

5. Запустить

```
cd /opt/mikroflow
docker compose up -d --build
docker compose ps          # все три сервиса Up (postgres healthy)
docker compose logs -f collector worker
```

6. Проверить, что данные идут

```
# сырые флоу — появляются в течение минуты-двух
docker compose exec postgres psql -U mikroflow -d mikroflow \
  -c "SELECT count(*) FROM flows_raw;"

# аренды DHCP и ARP (дают имена/MAC устройств)
docker compose exec postgres psql -U mikroflow -d mikroflow \
  -c "SELECT count(*) FROM dhcp_leases WHERE valid_to IS NULL;"
docker compose exec postgres psql -U mikroflow -d mikroflow \
  -c "SELECT count(*) FROM arp;"

# покрытие именами/MAC в агрегатах
docker compose exec postgres psql -U mikroflow -d mikroflow \
  -c "SELECT count(*) total, count(device_name) named, count(mac) macd FROM flows_hourly;"
```

Тайминги наполнения: `flows_raw` — сразу; `dhcp_leases / arp` — после первого опроса роутера (каждые 5 мин); `v_connections` — после первого полного часа (агрегатор идёт каждые 5 мин).

7. Анализ данных

Каждая строка `v_connections` — одно соединение: `device_name / mac / device_ip` — локальный хост, `remote_domain / remote_ip / remote_port` — с кем связь, `bytes` — суммарно в обе стороны. Время `hour` хранится в **UTC** (клиент, напр. DBEaver, показывает в твоём часовом поясе).

```
SELECT hour, device_name, mac, device_ip, remote_domain, remote_ip,
       remote_port, bytes, flow_count
FROM v_connections
WHERE hour >= now() - interval '7 days'
ORDER BY bytes DESC
LIMIT 100;
```

8. Имена устройств

Имя (`device_name`) выбирается по приоритету: **справочник** `device_alias` → **hostname из DHCP** → пусто. MAC берётся из DHCP или ARP.

Найти активные устройства без имени, которым стоит завести алиас:

```
SELECT DISTINCT mac, host(device_ip) AS ip
FROM v_connections
WHERE mac IS NOT NULL AND device_name IS NULL
ORDER BY ip;
```

Задать свои имена по MAC (регистр не важен):

```
INSERT INTO device_alias (mac, name) VALUES
('BC:24:11:DB:51:16', 'srv-backup'),
('BC:5F:F4:51:6B:F8', 'Ноутбук директора')
ON CONFLICT (mac) DO UPDATE SET name = EXCLUDED.name;
```

Новые часы подхватят алиасы автоматически. Чтобы перезаполнить именами уже посчитанную историю — сбрось водяной знак агрегатора (это идиоматично):

```
DELETE FROM agg_state;
```

9. Доступ к БД из DBeaver

Postgres опубликован только на **loopback сервера** (127.0.0.1:5432) — наружу не торчит. Подключайся через **SSH-туннель** (DBeaver умеет из коробки):

- вкладка **SSH**: галка «Use SSH Tunnel», Host = IP сервера, порт 22, твой ssh-логин и ключ/пароль;
- вкладка **Main**: Host = 127.0.0.1, Port = 5432, Database = mikroflow, User = mikroflow, Password = из .env.

10. Что учесть для прода

- **Файрвол сервера**: открой входящий **UDP 2055** только с IP роутера. API MikroTik (8728) — исходящий с сервера. Порт 5432 наружу не открывай.
- **Данные переживают перезапуск**: Postgres в volume pgdata, стек с restart: unless-stopped. Партии и миграции схемы применяются автоматически при старте; ретеншн (raw — 14 дней, hourly — 180 дней) чистит воркер.
- **Бэкап**: docker compose exec postgres pg_dump -U mikroflow mikroflow > backup.sql.
- **Объём**: при ~500 флоу/сек сырьё за 14 дней — сотни ГБ. Если диск ограничен, снизь MIKROFLOW_RAW_RETENTION_DAYS (напр. до 3–7). Следи за df -h.
- **Тюнинг под нагрузку**: при пропусках подними MIKROFLOW_RECV_BUFFER_BYTES и MIKROFLOW_BATCH_SIZE в .env.
- **Reverse-DNS**: домены CDN могут не резолвиться — принятое ограничение лёгкого пути (NetFlow без зеркалирования трафика).